

Technical Analysis using R

Assignment 5: Development Phase

This document provides clear instructions on how to evaluate the R script files generated for this assignment, along with brief explanations detailing the methodology used to calculate each technical indicator strictly using base R core functions.

Instructions for Execution

To run and evaluate the implemented functions, ensure all .R script files are located in your current R working directory.

Because some indicators rely on others (for instance, MACD depends on EMA, and StochRSI depends on both RSI and SMA), you must load all functions into your global environment before executing test datasets.

Use the following commands in your R console:

```
source("sma.R")
source("ema.R")
source("macd.R")
source("stdev.R")
source("linreg.R")
source("rsi.R")
source("stoch_rsi.R")
source("crossover.R")
source("crossunder.R")
```

Implementation Explanations

1. Simple Moving Average (SMA)

The `sma()` function iteratively computes the arithmetic mean of data chunks spanning the user-defined `period`. A vector of length `length(data) - period + 1` is instantiated to hold the resulting averages. The code strictly utilizes `sum()` within a loop, bypassing the need for rolling function libraries like `zoo` or `TTR`.

2. Exponential Moving Average (EMA)

The `ema()` function applies a higher weight to recent prices utilizing a calculated `multiplier` ($2 / (\text{period} + 1)$). The algorithm iteratively builds the array; the first data point acts as the baseline EMA, and subsequent points are derived via the recursive formula, ensuring no external dependencies.

3. Moving Average Convergence Divergence (MACD)

Relying heavily on the custom `ema()` function, `macd()` performs three rapid calculations to return a comprehensive list structure. It computes a short-term EMA, a long-term EMA, constructs the MACD

Implementation Explanations (Continued)

5. Linear Regression (linreg)

The `linreg()` function applies ordinary least squares logic directly. Utilizing index subsets defined by `regressionLength` and `regressionOffset`, it computes numeric sums and means to establish the formula's numerator and denominator. It ultimately returns the line's calculated slope, intercept, and a set of predicted values formatted as an R list.

6. Relative Strength Index (RSI)

To measure momentum, `rsi()` manually tracks point-to-point price differences via a custom iteration loop (substituting the native `diff()` for stricter core adherence). It compartmentalizes up-closes into a gains array and down-closes into a losses array. Following Wilder's smoothing method, it iteratively assesses the relative strength (RS) and translates it to an index of 0-100.

7. Stochastic RSI (StochRSI)

The `stoch_rsi()` function scales the outputs of `rsi()` utilizing a basic min-max normalization. Because of the leading `NA` values produced by smoothing buffers in the initial RSI algorithm, internal `na.rm=TRUE` arguments and index-matching safety measures are employed. It then utilizes the `sma()` function to deduce the %K and %D signal lines.

8. Crossover

Serving as an elementary trend-reversal tracker, `crossover()` accepts two equal-length arrays. It scans sequentially for instances where `arr1` overtakes `arr2` from a starting deficit. It produces a boolean/character vector logging "Up", "Down", or "None", enabling straightforward algorithmic trade triggers.

9. Crossunder

Mirroring the Crossover logic strictly for descending breaks, `crossunder()` flags instances where a dominant array dips below its companion line compared to the preceding period. It emits "True" only upon an active crossing, leaving other states as "False", which functions perfectly as an automated sell/short signal in subsequent trading rules.